Machine Learning for Finance

Unsupervised Learning - Dimension Reduction

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

Reducing the Dimension of Momentum

Q: How form a momentum trading signal using Dimension Reduction?

- Suppose we want to construct a momentum signal using a double moving-average crossover (or equivalently, short-horizon minus long-horizon returns).
- A key modeling choice is the definition of the *long* and *short* horizons.
- For example, the "long" horizon could plausibly be 11, 12, or 13 months, each yielding a slightly different signal.
- Rather than selecting a single horizon ex ante, we can compute momentum signals over *multiple* long horizons.
- Dimension Reduction provides a way to combine these correlated signals by reducing them to a single dominant "long-horizon" factor that captures the common variation.

Related work: Zakarmulin (2015), *Market Timing with Moving Averages*, applies similar ideas using moving averages of prices rather than returns.

Formalizing the Signal

A toy example that goes long(short) stocks with high(low) momentum

1. Compute "Long Return Returns" as $\tilde{R}^{Long} = \{R^{(13m)}, R^{(12m)}, R^{(11m)}\}$ and reduce to a signal score z_{i1}^{Long}
2. Repeat for z_{i1}^{Short}
3. Compute momentum as $Mom_i = z_{i1}^{Short} - z_{i1}^{Long}$

$$
\begin{aligned}
s_{i,t} &= \mathbf{1}\Big[p^{(+)}(X_{i,t}, Z_t)\Big] - \mathbf{1}\Big[p^{(-)}(X_{i,t}, Z_t)\Big] \\
p^{(+)} &= 1[Mom_i \geq q_{.75}(Mom)] \\
p^{(-)} &= 1[Mom_i \leq q_{.25}(Mom)]
\end{aligned}
$$

- $X_{i,t}$: returns on asset i
- Z_t: returns for all other assets

Dimension Reduction Techniques

Motivation for the dimension reduction: Reduce the number of dimensions of the dataset while retaining the most important information.

A couple types of Dimension Reduction:

- Principal Component Analysis (PCA)
- Autoencoders

Note: There are several other dimension reduction techniques that fall under the rubric of unsupervised learning.

Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

Reducing Dimension of Returns with PCA

1. Compute returns on N assets over p horizons (here $p = 13$):

$$R = \{R^{(13\text{m})}, R^{(12\text{m})}, \ldots, R^{(1\text{m})}\}, \qquad R \in \mathbb{R}^{N \times p}$$

2. Center each feature (column): for horizon j define

$$\bar{R}^{(j)} = \frac{1}{N} \sum_{i=1}^{N} R_i^{(j)}, \qquad \tilde{R}_{ij} = R_i^{(j)} - \bar{R}^{(j)}$$

   Write the centered matrix $\tilde{R} \in \mathbb{R}^{N \times p}$.

3. Covariance of centered returns:

$$\Sigma = \frac{1}{N-1} \tilde{R}^\top \tilde{R} \in \mathbb{R}^{p \times p}$$

4. Compute first principal direction (loading vector):

$$v_1 = \arg\max_{\|v\|=1} v^\top \Sigma v$$

where Lagrangian $\mathcal{L}(v, \lambda) = v^\top \Sigma v - \lambda(v^\top v - 1)$ gives the stationary condition

$$\Sigma v = \lambda v,$$

so v_1 is the eigenvector associated with the largest eigenvalue λ_1, and

$$\lambda_1 = v_1^\top \Sigma v_1$$

5. First principal component is $z_1 = \tilde{R} v_1 \in \mathbb{R}^N$, with score for asset i:

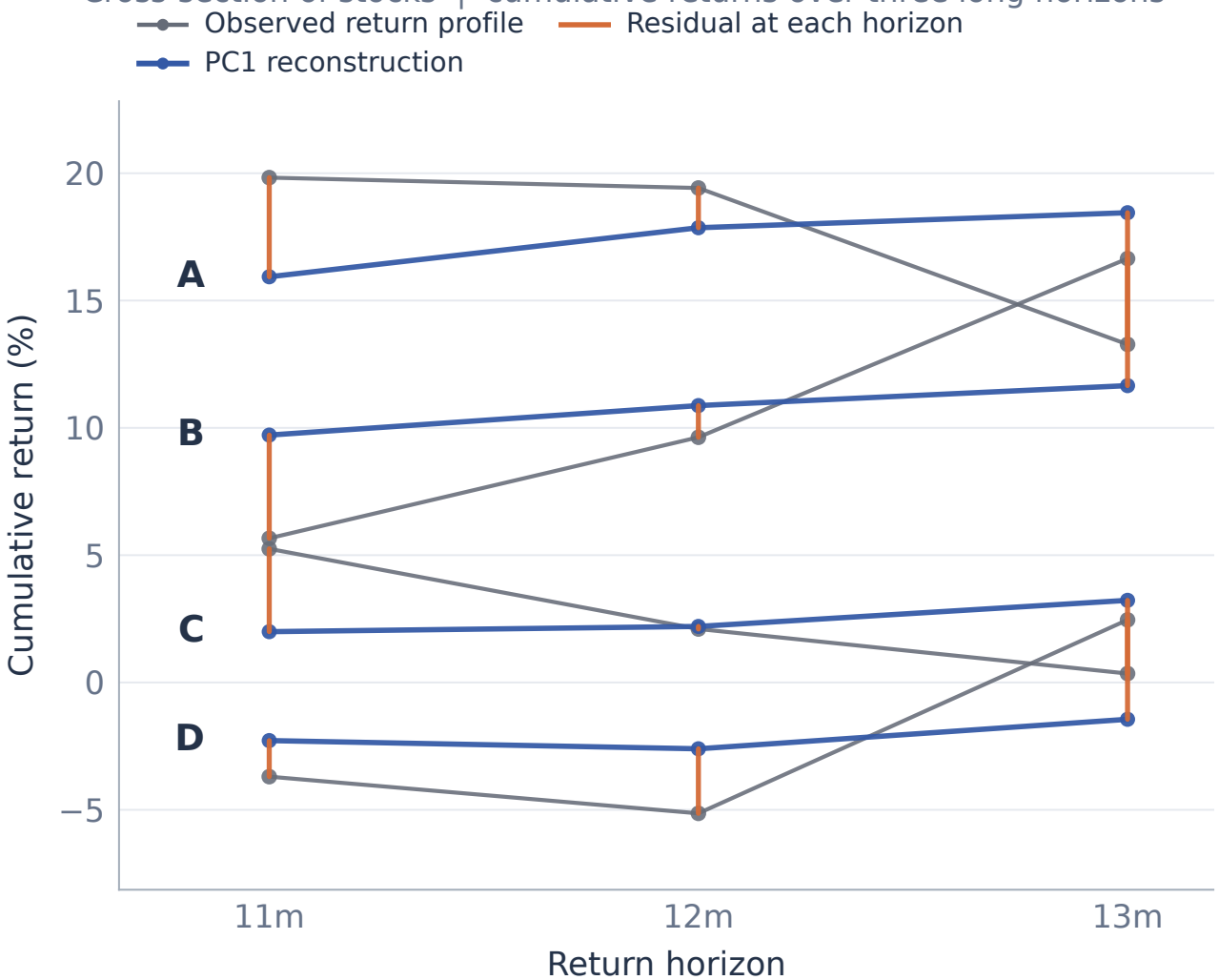
$$z_{i1} = \tilde{R}_{i\cdot}\, v_1$$

6. Rank-1 reconstruction (part explained by z_1): $\hat{R}^{(1)} = z_1 v_1^\top \in \mathbb{R}^{N \times p}$
7. Residual (what z_1 did not explain): $U_1 = \tilde{R} - \hat{R}^{(1)}$
8. The second principal direction v_2 is the first principal direction of the residual matrix U_1 (equivalently the eigenvector of Σ with the second largest eigenvalue). Repeat to get $v_3, \ldots, v_p$.

Loading, Scores, Reconstruction, Residuals

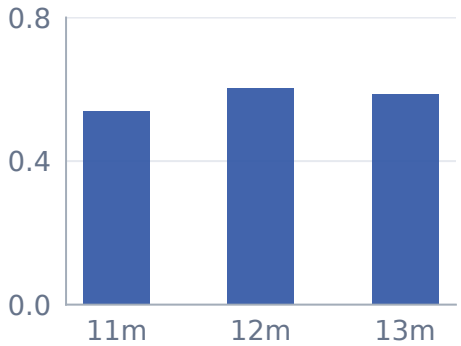
Extracting the first principal component

Cross-section of stocks | cumulative returns over three long horizons



Common loading vector

v_1: weights on 11m / 12m / 13m



PC1 scores by stock

$z_{A1} = +18.5$ pp

$z_{B1} = +6.9$ pp

$z_{C1} = -7.4$ pp

$z_{D1} = -15.4$ pp

$$\widehat{R}_i^{(1)} = \overline{R} + z_{i1} v_1$$

$$u_{i1} = R_i - \widehat{R}_i^{(1)}$$

Illustrative simulated returns. The mean profile is added back for the displayed reconstructions.

The geometry: four distinct PCA objects

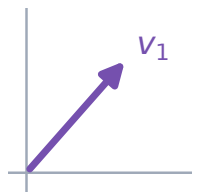
Two-feature illustration | 11- and 13-month centered returns



Four distinct objects

1 Loading vector

$v_1 = (+0.66, \ +0.75)$



Unit direction shared by stocks

2 Score

$z_{A1} = +13.4$ **pp**

Signed distance along PC1

3 Reconstruction

$\widehat{R}_A^{(1)} = z_{A1} v_1$

= (+8.9, +10.1) pp

4 Residual

$u_{A1} = \widetilde{R}_A - \widehat{R}_A^{(1)}$

= (+5.0, -4.4) pp

Blue arrow + orange arrow = observed stock A. The teal double arrow measures the signed PC1 score.

Two-feature PCA is estimated separately; the other figures use all three return horizons.

Building a Signal with PCA

Q: How build a momentum signal with PCA?

1. Compute "Long Return Returns" as $\tilde{R}^{Long} = \{R^{(13m)}, R^{(12m)}, R^{(11m)}\}$ and **collect the first PC score** $z_{i1}^{Long} = \tilde{R}_{i\cdot}^{Long} v_1^{Long}$
2. Repeat for z_{i1}^{Short}
3. Compute momentum $Mom_i = z_{i1}^{Short} - z_{i1}^{Long}$

$$
\begin{aligned}
s_{i,t} =& \mathbf{1}\left[p^{(+)}(X_{i,t}, Z_t)\right] - \mathbf{1}\left[p^{(-)}(X_{i,t}, Z_t)\right] \\
p^{(+)} =& 1[Mom_i \geq q_{.75}(Mom)] \\
p^{(-)} =& 1[Mom_i \leq q_{.25}(Mom)]
\end{aligned}
$$

- $X_{i,t}$: returns on asset i
- Z_t: returns for all other assets

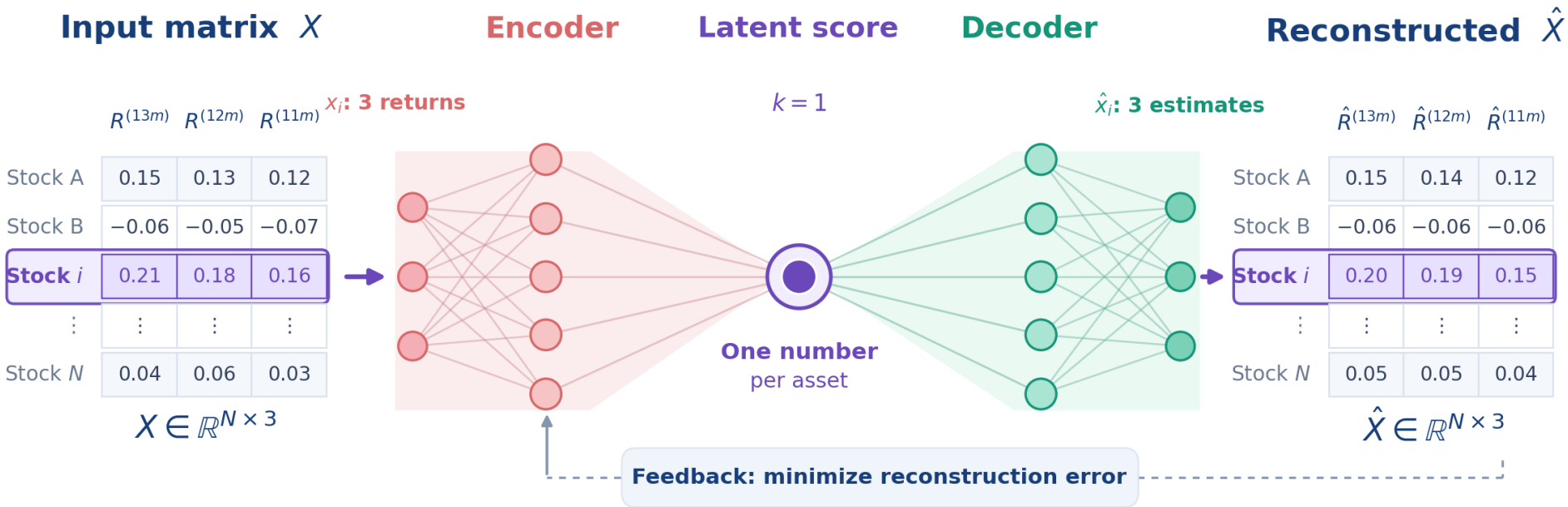
Outline

1. Dimension Reduction
   - PCA
   - Autoencoders

Autoencoder Intuition

Autoencoders: compressing momentum returns

A single learned score summarizes three related return horizons — and is trained to reconstruct them.



Reading the input matrix

Each row = one stock (one observation)

Each column = one return horizon

More return horizons could be included.

Why a bottleneck?

$$x_i\ (3) \longrightarrow z_i\ (1) \longrightarrow \hat{x}_i\ (3)$$

Shared network weights are used for every stock.

How is it trained?

$$\min_{\theta,\phi} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2$$

No target labels: reconstruct the inputs.

How Autoencoders Work

Goal: Learn a low-dimensional representation that retains information needed to reconstruct the original features.

Architecture:

$$x_i \in \mathbb{R}^p \xrightarrow{\text{Encoder } f_\theta} z_i \in \mathbb{R}^k \xrightarrow{\text{Decoder } g_\phi} \hat{x}_i \in \mathbb{R}^p, \qquad k < p.$$

Optimization problem (unsupervised):

$$\min_{\theta,\phi} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(x_i, g_\phi(f_\theta(x_i)))$$

- $z_i = f_\theta(x_i)$: *latent code* (compressed representation).
- $\hat{x}_i = g_\phi(z_i)$: reconstruction of x_i.
- $\mathcal{L}(x_i, \hat{x}_i)$: reconstruction loss; smaller is better.
- θ, ϕ: encoder and decoder parameters, learned *jointly*.
- No separate target labels: the network learns to reconstruct its input.

Reconstruction Loss Functions

Choice of loss: What kind of reconstruction error matters?

Loss	**Typical data**	**Main consideration**
MSE	Continuous	Penalizes large errors heavily.
MAE	Continuous	Less sensitive to extreme errors.
Huber	Continuous	Quadratic near zero; linear for large errors.
Cross-entropy	Binary (0/1)	Reconstructs event probabilities.

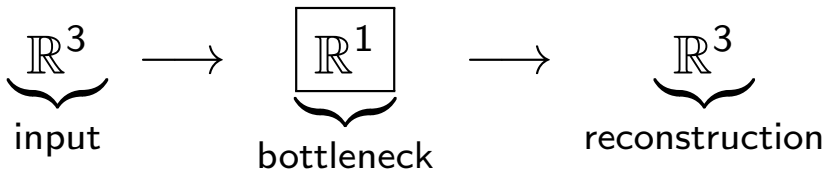
Mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(x_i, \hat{x}_i) = \frac{1}{p} \sum_{j=1}^{p} (x_{ij} - \hat{x}_{ij})^2$$

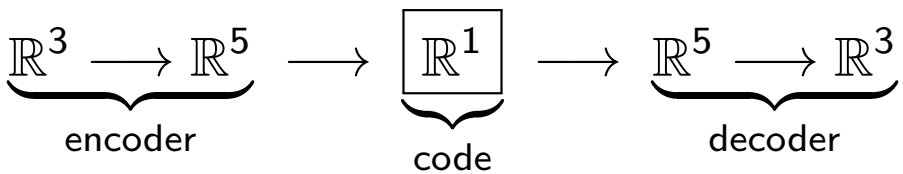
- **Our momentum example:** MSE is a natural starting point for returns.
- **Preprocessing:** Feature scales matter under MSE; consider standardizing.

Autoencoder Architecture

Start with a simple $3 \to 1 \to 3$ autoencoder:



Add intermediate hidden layers:



- **Hidden layer:** An intermediate transformation of the features.
- **Depth:** Additional layers can model more complex relationships *when paired with nonlinear activations*; they also raise overfitting risk.
- **Bottleneck ($k < p$):** Limits how much information passes from encoder to decoder, encouraging compression.

Smaller k imposes more compression, but may worsen reconstruction.

Encoder and Decoder Mappings

Example: One intermediate hidden layer on each side.

Encoder: f_θ

$$h_i = \rho_e(W_1 x_i + b_1),$$
$$z_i = W_2 h_i + b_2.$$

Decoder: g_ϕ

$$q_i = \rho_d(V_1 z_i + c_1),$$
$$\hat{x}_i = \tau(V_2 q_i + c_2).$$

- W, V are *weight matrices*; b, c are *biases*.
- ρ_e, ρ_d are hidden-layer activations; τ is the final output activation.
- We *choose* the number of layers, widths, and activations; training *learns* the weights and biases.
- The decoder need not mirror the encoder or be its mathematical inverse.
- For signed returns, a **linear output activation** ($\tau(u) = u$) is a natural choice.

Activation Functions

Nonlinear activations allow nonlinear encoder/decoder mappings.

Activation	Definition	Typical use
Linear	$a(u) = u$	PCA benchmark; unrestricted output.
ReLU	$a(u) = \max(0, u)$	Common nonlinear hidden layers.
tanh	$a(u) = \tanh(u)$	Bounded, zero-centered hidden values.
ELU	$a(u) = \begin{cases} u, & u > 0 \\ \alpha(e^u - 1), & u \leq 0 \end{cases}$	Nonlinear hidden layers allowing negative values.
Sigmoid	$a(u) = (1 + e^{-u})^{-1}$	Probability output for binary features.

- **Linear at every layer:** Even a deep network is only an affine mapping.
- **Linear at the output only:** Hidden layers may still be nonlinear; useful for signed, continuous returns.

Autoencoders and PCA

PCA (k components; centered observations):

$$z_i^{\mathrm{PCA}} = V_k^\top (x_i - \bar{x}), \qquad \hat{x}_i^{\mathrm{PCA}} = \bar{x} + V_k z_i^{\mathrm{PCA}}.$$

Linear autoencoder (illustrative parameterization):

$$z_i^{\mathrm{AE}} = A(x_i - \bar{x}), \qquad \hat{x}_i^{\mathrm{AE}} = \bar{x} + B z_i^{\mathrm{AE}}.$$

$$A \in \mathbb{R}^{k \times p}, \qquad B \in \mathbb{R}^{p \times k}$$

where A and B are the learned encoder and decoder weight matrices, respectively.

- With MSE loss and a k-dimensional bottleneck, a globally optimized linear autoencoder can recover the same *optimal reconstruction subspace* as PCA.
- The learned weights and latent codes need not equal PCA loadings and scores.
- Nonlinear activations permit the autoencoder to learn *nonlinear* low-dimensional structure.
- More flexibility does *not* guarantee better out-of-sample reconstruction or investment signals.

Building a Signal with Autoencoders

Q: How build a momentum signal using autoencoders?

1. **Long horizons:** Construct $\widetilde{R}_i^{\text{Long}} = (\widetilde{R}_i^{(13m)}, \widetilde{R}_i^{(12m)}, \widetilde{R}_i^{(11m)})$ and obtain $z_{i,t}^{\text{Long}}$ from an encoder with $k = 1$.
2. **Short horizons:** Repeat for $z_{i,t}^{\text{Short}}$.
3. **Make scores comparable:** Orient each score so higher values indicate stronger corresponding-horizon returns; then convert to cross-sectional percentile ranks:

$$u_{i,t}^{H} = \text{PctRank}_t\left(a_H z_{i,t}^{H}\right), \quad H \in \{\text{Short}, \text{Long}\}, \quad a_H \in \{-1, +1\}.$$

4. **Momentum:** $Mom_{i,t} = u_{i,t}^{\text{Short}} - u_{i,t}^{\text{Long}}$.

$$s_{i,t} = \mathbf{1}[Mom_{i,t} \geq q_{.75}(Mom_t)] - \mathbf{1}[Mom_{i,t} \leq q_{.25}(Mom_t)]\,.$$

Fix each sign a_H using training data; rank within date t over the same asset universe.